Guide démo live — Demo 1 + Demo 2 combinées

Public : client / décideur / audience technique (Sopra Steria) **Durée cible :** 15–20 minutes **Principe :** pour chaque étape, on montre les **deux faces** : - le **YAML dans la console OpenShift** (ce que le GitOps voit) - le **résultat dans le dashboard OpenShift AI** (ce que l'utilisateur voit)

URLs à garder ouvertes (3 onglets) :

Onglet	URL	Compte
Console OpenShift	`https://console-openshift-console.<domaine-cluster>`	admin
Dashboard RHOAI (admin)	`https://rhods-dashboard-redhat-ods-applications.<domaine-cluster>`	admin
Dashboard RHOAI (user)	Même URL, fenêtre privée	`maas-demo2-user`

Avant d'entrer en scène (5 min)

```
make demo1-verify          # 11 passed
make demo2-verify          # All checks passed
```

Vérifier que les 2 clés API fonctionnent (HTTP 200). Si elles sont mortes (403), les recréer depuis le dashboard `maas-demo2-user` → Gen AI studio → API keys → Create API key.

Fil rouge (à dire une fois, 30 s)

```
Console OpenShift (YAML)          Dashboard OpenShift AI (clics)
        │                                  │
        ├─ LLMInferenceService ──────── Gen AI studio → AI asset endpoints (modèle visib
        ├─ MaaSModelRef ─────────────── Gen AI studio → AI asset endpoints → badge MaaS
        ├─ MaaSSubscription ─────────── Settings → MaaS governance → Subscriptions (admi
        │                               Gen AI studio → API keys → Subscriptions (user)
        ├─ MaaSAuthPolicy ──────────── Settings → MaaS governance → Authorization polici
        │                               Gen AI studio → API keys → Create API key (user)
        └─ Gateway ──────────────────── curl /v1/chat/completions → 200
```

> « Pour chaque pièce de la chaîne, on va vous montrer le YAML Kubernetes réel — c'est ça que votre pipeline GitOps versionne — et le résultat tel qu'il apparaît dans l'interface utilisateur. »

PARTIE 1 — Le modèle natif (Demo 1)

Étape 1 — LLMInferenceService (3 min)

Console OpenShift : 1. Menu gauche → **Search** → Resources : `LLMInferenceService` 2. Namespace : `rhoai-maas-demo` 3. Clique sur `demo-simulator` → onglet **YAML** 4. Montre les champs clés : - `kind: LLMInferenceService` (le chemin natif llm-d, pas legacy KServe) - `spec.model.name: facebook/opt-125m` (le nom exposé via l'API) - `spec.router.gateway.refs` → pointe vers `maas-default-gateway` 5. Onglet **Details** → montre `Ready: True`

> « C'est le manifeste qui déploie le modèle. C'est du YAML standard, versionnable en Git. Sur un vrai cluster, le champ uri pointerait vers vos poids sur S3 ou HuggingFace. »

↔ **Bascule vers le Dashboard RHOAI (admin) :** 1. **Gen AI studio** → **AI asset endpoints** → onglet Models → projet `rhoai-maas-demo` 2. Montre `demo-simulator` avec les endpoints (External + Internal)

> « Le chemin natif llm-d (nouveau dans RHOAI 3.5) apparaît dans Gen AI studio, pas dans Projects → Deployments qui ne montre que les déploiements legacy KServe/vLLM. Gen AI studio, c'est la vue MaaS — c'est ici que l'admin et l'utilisateur voient les modèles publiés. »
>
> ⚠️ **Ne pas aller dans Projects → Model Deployments** pour chercher `demo-simulator` — il n'y sera pas. C'est normal : cette vue ne liste que les `InferenceService` legacy, pas les `LLMInferenceService` natifs.

↔ **Montrer le formulaire de déploiement (le geste admin en clics) :**

> ⚠️ NE PAS utiliser « Edit model deployment » sur l'existant — le formulaire sera vide car le modèle a été déployé par YAML. Utiliser « **Deploy model** » (nouveau) à la place.

1. Reste dans le projet `rhoai-maas-demo` → clique le bouton **Deploy model**
2. **Page 1 — Model details** (remplir en live, 15 sec) : - **Model location** → sélectionne **URI** - **URI** → tape `hf://sshleifer/tiny-gpt2` - **Model type** → `Generative AI model (Example, LLM)` (déjà sélectionné) - **Name** → tape `demo-simulator`
3. Clique **Next**

4. **Page 2 — Model deployment** : - Montre les deux options de déploiement :
   - ~~Single-node with KServe~~ ← « C'est l'ancien chemin legacy »
   - ☑ **Distributed inference with llm-d** ← « C'est le chemin **natif** qu'on utilise »
   - **NE PAS cocher** « Use legacy deployment method »
5. Clique **Next**
6. **Page 3 — Advanced settings** : - Descends jusqu'à **Model availability** - Montre la case ☑ **Publish as MaaS** ← « C'est ce qui crée automatiquement le MaaSModelRef »
7. → **Cancel** ⛔ (ne surtout pas cliquer Deploy — le GPU est déjà pris)

> 💬 « Voilà le geste complet pour un admin qui préfère les clics au YAML : URI du modèle, type Generative AI, déploiement llm-d, et la case Publish as MaaS. Cette case fait exactement la même chose que le MaaSModelRef qu'on a vu côté YAML — deux chemins, même résultat. »

Étape 2 — MaaSModelRef, la publication (2 min)

Console OpenShift : 1. Search → Resources : `MaaSModelRef` → namespace `rhoai-maas-demo` 2. Clique sur `demo-simulator` → onglet **YAML** 3. Montre : `yaml spec: modelRef: kind: LLMInferenceService name: demo-simulator` 4. Onglet **Details** → montre `phase: Ready`, conditions `GovernancePaired`, `RuntimeHealthy`

> « C'est la pièce qui publie le modèle dans le catalogue MaaS. Un seul lien : "ce LLMInferenceService entre dans MaaS". C'est l'équivalent déclaratif de la case Publish as MaaS. »

↔ **Bascule vers le Dashboard RHOAI (admin) :** 1. **Gen AI studio** → **AI asset endpoints** → onglet Models 2. Cherche `demo-simulator` → clique **View** 3. Montre le badge **Model as a Service** et l'endpoint HTTPS

> « Le modèle est maintenant dans le catalogue gouverné. Les utilisateurs le verront quand ils auront un abonnement. »

PARTIE 2 — La gouvernance (Demo 2)

Étape 3 — MaaSSubscription, le quota (2 min)

🔵 **Console OpenShift (YAML technique) :** 1. Search → Resources : `MaaSSubscription` → namespace **`models-as-a-service`** 2. Clique sur `maas-demo2-subscription` → onglet **YAML** 3. Montre : `yaml spec: owner: groups: - name: maas-demo2-developers # ← quel groupe modelRefs: -`

`name: demo-simulator # ← quel modèle tokenRateLimits: - limit: 2000 # ← combien window: 1m # ← par minute`

> « La Subscription, c'est le COMBIEN : quel groupe a droit à combien de tokens par minute sur quel modèle. Notez le namespace : models-as-a-service, pas celui du modèle — c'est le namespace du tenant MaaS. »

🟢 **Bascule vers le Dashboard RHOAI (admin) — vue admin MaaS :** 1. **Settings** → **MaaS governance** → onglet **Subscriptions** 2. Montre `maas-demo2-subscription` : statut **Active**, groupes, modèles, token limits 3. Clique sur la subscription → montre le détail (groupes, modèles, limites, priorité)

> « Côté admin, voici la vue gouvernance dédiée MaaS : toutes les subscriptions, les policies, en un seul endroit. L'admin peut aussi les créer et les modifier depuis cette interface, pas seulement en YAML. »

🟣 **Bascule vers le Dashboard RHOAI (session `maas-demo2-user`) — vue utilisateur :** 1. Gen AI studio → API keys → onglet **Subscriptions** 2. Montre l'abonnement `maas-demo2-subscription` avec le quota `2,000 / 1 minute` 3. Développe la ligne pour voir le détail

> « Et côté utilisateur : il voit son abonnement et sa limite. Trois vues du même objet — YAML, admin dashboard, user dashboard — chacune adaptée à son rôle. »

Étape 4 — MaaSAuthPolicy, l'autorisation (2 min)

🔵 **Console OpenShift (YAML technique) :** 1. Search → Resources : `MaaSAuthPolicy` → namespace `models-as-a-service` 2. Clique sur `maas-demo2-access` → onglet **YAML** 3. Montre : `yaml spec: subjects: groups: - name: maas-demo2-developers # ← QUI a le droit modelRefs: - name: demo-simulator # ← sur QUEL modèle`

> « L'AuthPolicy, c'est le QUI : quel groupe est autorisé à appeler quel modèle via la gateway. Sans cette pièce, même avec une Subscription, l'utilisateur prend un 403. Les deux sont nécessaires ensemble. »

🟢 **Bascule vers le Dashboard RHOAI (admin) — vue admin MaaS :** 1. **Settings** → **MaaS governance** → onglet **Authorization policies** 2. Montre `maas-demo2-access` : statut **Active**, groupes autorisés, modèles 3. Clique pour voir le détail

> « L'admin voit toutes les policies d'autorisation ici. Il peut aussi en créer, modifier ou supprimer directement depuis cette interface. »

🟣 **Bascule vers le Dashboard RHOAI (session `maas-demo2-user`) — preuve que ça marche :** 1. Gen AI studio → API keys → **Create API key** 2. Montre que le sélecteur de Subscription est **actif** (pas grisé) → preuve que l'AuthPolicy fonctionne 3. Créer la clé → révéler une fois → copier

> « La preuve côté utilisateur : il peut créer une clé. Sans l'AuthPolicy, ce sélecteur serait désactivé — "No subscriptions available". »

Étape 5 — Inférence réelle (1 min)

Terminal :

```
make demo2-test-inference
```

> « Appel OpenAI-compatible réel, avec la clé de l'utilisateur final. GET /v1/models → 200, POST /v1/chat/completions → 200 avec une vraie réponse. »

Étape 6 — Tests négatifs, la preuve que la gouvernance marche (2 min)

Terminal :

```
make demo2-test-negative
```

Pendant que ça tourne (~90 s) :

> « Trois tests automatisés : on retire la policy → 403. On retire la subscription de cette clé → 403. On épuise un quota minuscule dédié → 429. Tout est restauré automatiquement à la fin. »

Étape 7 — Observabilité admin (1 min)

Terminal :

```
make demo2-check-usage
```

> « Les compteurs Prometheus de Kuadrant/Limitador — authorized_calls, limited_calls — correspondent exactement aux appels qu'on vient de faire. Limite honnête : c'est agrégé par modèle, pas par utilisateur. »

Étape 8 — Révoquer / régénérer (1 min)

Dashboard RHOAI (session `maas-demo2-user`) : 1. Kebab (⋮) sur une clé → **Revoke** → confirmer 2. Montrer la clé disparue

> « Révocation immédiate. Ce sandbox renvoie 403, pas le 401 de la doc — c'est un constat réel, pas une erreur. »

Clôture (30 s)

> « Récap : chaque pièce est un CR Kubernetes que votre pipeline GitOps peut versionner. L'interface OpenShift AI montre le résultat utilisateur, la console OpenShift montre le YAML technique. Les deux faces de la même réalité. Et toute cette démo est rejouable en une commande : `make demo1-all && make demo2-all`. »

Ce qu'il ne faut PAS faire en live

❌ À éviter	Pourquoi
Dire "401" pour une clé révoquée	Le vrai code est **403**
Promettre un suivi d'usage par utilisateur	Limitador agrège par modèle seulement
Montrer "AI asset endpoints" pour créer une clé temporaire	Ça renvoie un 403
Laisser tourner `demo2-test-negative` sans prévenir	~90s, le client peut croire que c'est planté
Cliquer "Deploy" dans le wizard au lieu de "Cancel"	Le GPU est déjà pris, ça échouerait
Utiliser "Edit model deployment" sur demo-simulator	Le formulaire sera vide (modèle déployé par YAML, pas par le wizard) — utiliser "Deploy model" (nouveau) à la place
Chercher demo-simulator dans Projects → Model Deployments	Les `LLMInferenceService` natifs n'apparaissent pas ici — utiliser **Gen AI studio → AI asset endpoints**

Aide-mémoire (post-it)

```
PARTIE 1 — LE MODÈLE
  Console: Search → LLMInferenceService → demo-simulator (YAML + Ready)
  Dashboard: Gen AI studio → AI asset endpoints → demo-simulator (endpoints)
  Dashboard: Deploy model (NOUVEAU, pas Edit) → URI + llm-d + Publish as MaaS → Cancel
  Console: Search → MaaSModelRef → demo-simulator (YAML + GovernancePaired)
  Dashboard: Gen AI studio → AI asset endpoints → badge MaaS

PARTIE 2 — LA GOUVERNANCE
  Console: Search → MaaSSubscription → ns models-as-a-service (YAML: quota)
  Dashboard admin: Settings → MaaS governance → Subscriptions (Active, quotas)
  Dashboard user: Gen AI studio → API keys → Subscriptions (vue utilisateur)
  Console: Search → MaaSAuthPolicy → ns models-as-a-service (YAML: groups)
  Dashboard admin: Settings → MaaS governance → Authorization policies (Active)
  Dashboard user: Create API key → sélecteur actif → créer
  Terminal: make demo2-test-inference → 200
  Terminal: make demo2-test-negative → 403 / 403 / 429
  Terminal: make demo2-check-usage → compteurs Limitador
  Dashboard user: Revoke → 403
```